

DevOps Interview Preparation

Docker • Kubernetes • Terraform

From Scratch to Associate Software Engineer

Complete L1 & L2 Interview Guide

Target Role	Associate Software Engineer / Junior DevOps
Interview Level	L1 (Screening) & L2 (Technical)
Topics Covered	Docker, Kubernetes, Terraform
Format	Concepts + Commands + Q&A

Table of Contents

PART 1	DOCKER	
1.1	Introduction to Containers and Docker	3
1.2	Docker Architecture	4
1.3	Installing Docker	5
1.4	Docker Images	5
1.5	Docker Containers	6
1.6	Dockerfile	7
1.7	Docker Volumes and Storage	8
1.8	Docker Networking	9
1.9	Docker Compose	10
1.10	Docker Registry & Hub	11
1.11	Best Practices	11
1.12	Docker Interview Q&A (L1 + L2)	12
PART 2	KUBERNETES	
2.1	Why Kubernetes?	16
2.2	Kubernetes Architecture	16
2.3	kubectl - The CLI	18
2.4	Pods	18
2.5	ReplicaSets and Deployments	19
2.6	Services	20
2.7	ConfigMaps and Secrets	21
2.8	Namespaces	22
2.9	Persistent Volumes	22
2.10	Ingress	23
2.11	Probes and Health Checks	23
2.12	Kubernetes Interview Q&A (L1 + L2)	24
PART 3	TERRAFORM	
3.1	Infrastructure as Code (IaC)	28
3.2	What is Terraform?	28
3.3	Terraform Workflow	29
3.4	Providers and Resources	30
3.5	Variables, Outputs, Locals	31
3.6	Terraform State	32
3.7	Modules	33
3.8	Workspaces	34
3.9	Provisioners and Lifecycle	34
3.10	Best Practices	35

3.11	Terraform Interview Q&A (L1 + L2)	36
PART 4	INTERVIEW STRATEGY	
4.1	How to Answer L1 vs L2 Questions	40
4.2	Scenario-Based Cross-Topic Questions	40
4.3	Final Tips	42

PART 1: DOCKER

Docker is the most widely adopted container platform. It is almost always the first topic you will be tested on in a DevOps or backend interview. This section takes you from the absolute basics to the level where you can comfortably answer L1 and L2 questions.

1.1 Introduction to Containers and Docker

A **container** is a lightweight, standalone, executable package that includes everything needed to run an application: code, runtime, system tools, libraries, and settings. Containers share the host machine's OS kernel, making them much more efficient than virtual machines.

Docker is an open-source platform that automates the deployment of applications inside containers. It packages an application and its dependencies into a container that can run anywhere Docker is installed.

Containers vs Virtual Machines

Aspect	Virtual Machines	Containers
OS	Full guest OS per VM	Share host OS kernel
Size	GBs (heavy)	MBs (lightweight)
Boot time	Minutes	Seconds
Isolation	Hardware-level (strong)	Process-level (weaker)
Performance	Lower (hypervisor)	Near-native
Density	Few per host	Hundreds per host

Why Docker?

- **Portability:** Build once, run anywhere (laptop, on-prem, cloud).
- **Consistency:** Eliminates the "it works on my machine" problem.
- **Isolation:** Each container runs independently.
- **Efficiency:** Uses fewer resources than VMs.
- **Faster delivery:** CI/CD pipelines benefit immensely.
- **Microservices ready:** Perfect for modern distributed architectures.

1.2 Docker Architecture

Docker uses a **client-server architecture**. The Docker client talks to the Docker daemon (server), which does the heavy lifting of building, running, and managing containers.

Key Components

- **Docker Daemon (dockerd):** Background service that manages Docker objects (images, containers, networks, volumes).
- **Docker Client (docker):** The CLI tool you use to interact with Docker via REST API.
- **Docker Registry:** Storage for Docker images (e.g., Docker Hub, AWS ECR, GCR).

- **Docker Objects:** Images, Containers, Volumes, Networks, Plugins.
- **Container Runtime:** Low-level component (runc, containerd) that actually runs containers.

Image vs Container

An **image** is a read-only template (a blueprint). A **container** is a running instance of an image. Think class vs object in OOP — one image can create many containers.

1.3 Installing Docker

On Linux (Ubuntu/Debian):

```
# Install Docker Engine
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io

# Verify
docker --version
docker run hello-world
```

On Windows and macOS, install **Docker Desktop**, which bundles Docker Engine, CLI, Compose, and a small Linux VM.

1.4 Docker Images

An image is a layered, read-only template. Each instruction in a Dockerfile creates a new layer. Layers are cached and reused across images.

Common Image Commands

```
docker pull nginx:latest      # Download image from registry
docker images                 # List local images
docker rmi nginx:latest       # Remove image
docker image inspect nginx    # Detailed metadata
docker image prune            # Remove unused images
docker tag myapp:1.0 user/myapp:1.0 # Tag for registry
docker push user/myapp:1.0    # Push to registry
```

Image Layers

Each Dockerfile instruction (FROM, RUN, COPY, etc.) creates a layer. Layers are stacked and union-mounted. If a layer doesn't change, Docker reuses the cached version — this is why ordering Dockerfile instructions matters for build speed.

1.5 Docker Containers

Lifecycle Commands

```
# Run a container
docker run -d --name web -p 8080:80 nginx
#   -d : detached (background)
#   --name : custom name
#   -p host:container : port mapping

docker ps                # List running containers
```

```

docker ps -a          # List ALL (including stopped)
docker stop web       # Stop gracefully (SIGTERM, then SIGKILL)
docker start web      # Start a stopped container
docker restart web    # Stop + start
docker rm web         # Remove (must be stopped)
docker rm -f web      # Force remove
docker logs -f web    # Stream logs
docker exec -it web sh # Open shell inside container
docker inspect web    # Full JSON details
docker stats          # Live resource usage

```

Common Run Flags

Flag	Meaning
-d	Detached mode (background)
-it	Interactive + TTY (for shells)
-p host:container	Publish port
-e KEY=value	Set environment variable
-v host:container	Mount volume / bind mount
--name NAME	Assign a name
--rm	Auto-remove when stopped
--network NAME	Connect to a specific network
--restart policy	Restart policy (no/always/on-failure/unless-stopped)
--memory 512m	Limit RAM
--cpus 1.5	Limit CPU

1.6 Dockerfile

A **Dockerfile** is a text file with instructions to build an image. Each instruction creates a layer.

Common Instructions

Instruction	Purpose
FROM	Base image (must be first)
WORKDIR	Set working directory inside container
COPY	Copy files from host into image
ADD	Like COPY but also handles URLs and tar extraction
RUN	Execute command at build time (creates layer)
CMD	Default command when container starts (can be overridden)
ENTRYPOINT	Main executable (harder to override)
ENV	Set environment variable
EXPOSE	Document which port the app listens on (informational)
VOLUME	Declare a volume mount point
USER	Switch user for subsequent instructions
ARG	Build-time variable
LABEL	Add metadata

Example: Node.js App Dockerfile

```
# Stage 1: Build
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

# Stage 2: Runtime (smaller final image)
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
ENV NODE_ENV=production
EXPOSE 3000
USER node
CMD ["node", "dist/server.js"]
```

CMD vs ENTRYPOINT (Common Interview Question)

CMD provides default arguments that can be overridden at runtime. **ENTRYPOINT** defines the command that always runs; arguments after the image name are appended to ENTRYPOINT. Use ENTRYPOINT for executable-style containers, CMD for flexible defaults. They are often combined: ENTRYPOINT sets the binary, CMD sets default args.

Building Images

```
docker build -t myapp:1.0 . # . = build context
docker build -t myapp:1.0 -f Dockerfile.prod .
docker build --no-cache -t myapp:1.0 . # Ignore cache
docker build --build-arg VERSION=1.2 -t myapp .
```

1.7 Docker Volumes and Storage

Containers are **ephemeral** — data written inside them disappears when the container is removed. To persist data, use volumes.

Three Storage Options

- **Volumes (recommended):** Managed by Docker, stored in `/var/lib/docker/volumes/`. Portable across environments.
- **Bind mounts:** Mount a host directory directly into the container. Good for development.
- **tmpfs mounts:** Stored in host memory only. Good for sensitive temporary data.

```
# Named volume
docker volume create mydata
docker volume ls
docker volume inspect mydata
docker run -v mydata:/var/lib/mysql mysql

# Bind mount
docker run -v $(pwd)/app:/app node:18

# Modern syntax (preferred)
docker run --mount type=volume,source=mydata,target=/data mysql
```

```
docker run --mount type=bind,source=$(pwd),target=/app node:18
```

1.8 Docker Networking

Docker provides several network drivers to connect containers to each other and to the outside world.

Network Types

Driver	Use Case
bridge (default)	Containers on the same host can talk to each other
host	Container shares host's network stack (no isolation)
none	No networking; fully isolated container
overlay	Multi-host networking (Swarm/Kubernetes)
macvlan	Container gets its own MAC address on the LAN

```
docker network ls          # List networks
docker network create mynet # Create custom bridge
docker network inspect mynet # Details
docker network connect mynet web # Attach a container
docker run --network mynet --name api node:18
```

```
# On custom bridges, containers can find each other by NAME (DNS)
# So 'api' container can reach 'db' container at hostname: db
```

1.9 Docker Compose

Docker Compose defines and runs multi-container applications with a single YAML file (*docker-compose.yml*) and one command.

Example: Web + DB Stack

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8080:3000"
    environment:
      - DB_HOST=db
    depends_on:
      - db
    restart: unless-stopped

  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Key Compose Commands


```
docker compose up -d          # Start everything in background
docker compose down          # Stop and remove containers + networks
docker compose down -v       # Also remove volumes
docker compose ps            # Status of services
docker compose logs -f web   # Follow logs of one service
docker compose build         # Build/rebuild images
docker compose exec web sh   # Shell into a running service
```

1.10 Docker Registry & Hub

A **registry** stores Docker images. **Docker Hub** is the default public registry. Private registries include AWS ECR, Google GCR, Azure ACR, GitHub Container Registry, and self-hosted Harbor.

```
docker login                  # Login to Docker Hub
docker login myregistry.example.com # Private registry
docker tag myapp:1.0 username/myapp:1.0
docker push username/myapp:1.0
docker pull username/myapp:1.0
```

1.11 Best Practices

- Use **specific image tags** (e.g., *node:18.17.0-alpine*), not *latest* in production.
- Use **multi-stage builds** to keep final images small.
- Use **.dockerignore** to exclude unnecessary files (*node_modules*, *.git*, *secrets*).
- Order Dockerfile instructions from **least to most frequently changing** to maximize cache reuse.
- Run containers as a **non-root user** (USER instruction).
- Use **minimal base images** like *alpine* or *distroless*.
- Never bake **secrets** into images — use env vars, secret managers, or build-time secrets.
- Use **health checks** with the HEALTHCHECK instruction.
- Set explicit **resource limits** (*--memory*, *--cpus*) in production.
- **One process per container** — keep concerns separated.

1.12 Docker Interview Q&A;

L1 — Screening Level

Q: What is Docker?

A: Docker is an open-source platform to develop, ship, and run applications inside lightweight, portable containers. It packages an app with all its dependencies so it runs reliably on any environment that has Docker installed.

Q: What is the difference between a container and a virtual machine?

A: VMs virtualize hardware and run a full guest OS, making them heavy (GBs) and slow to boot. Containers virtualize the OS — they share the host kernel and only package the app + its libraries, making them lightweight (MBs) and starting in seconds.

Q: What is the difference between a Docker image and a Docker container?

A: An image is a read-only template (the blueprint) with the application and its dependencies. A container is a running instance of an image. From one image you can create many containers.

Q: What is a Dockerfile?

A: A text file containing a set of instructions that Docker uses to build an image automatically. Each instruction (FROM, COPY, RUN, CMD, etc.) creates a layer in the resulting image.

Q: What is Docker Hub?

A: Docker Hub is the default public registry where Docker images are stored and shared. You can push your own images and pull official images (nginx, mysql, node, etc.) from it.

Q: Difference between COPY and ADD?

A: Both copy files into the image. COPY only does a straightforward file copy. ADD does that plus extra magic — it can extract local tar files and download from URLs. Best practice is to use COPY unless you specifically need ADD's extra features.

Q: What does the -p flag do in docker run?

A: It publishes a container port to the host. `-p 8080:80` maps port 80 inside the container to port 8080 on the host, so traffic to host:8080 reaches the container.

Q: How do you stop and remove a container?

A: `docker stop <name>` stops it gracefully (SIGTERM then SIGKILL after a timeout). `docker rm <name>` removes it. Use `docker rm -f` to do both at once.

Q: What is Docker Compose used for?

A: Docker Compose is used to define and run multi-container applications using a single YAML file. With one command (`docker compose up`) you can spin up an entire stack — for example, a web app, a database, and a cache — with the right networks and volumes.

Q: Where does Docker store data by default?

A: On Linux, in `/var/lib/docker`. Container layers, images, volumes, and networks all live under that directory.

L2 — Technical Level

Q: Explain Docker architecture.

A: Docker uses a client-server architecture. The Docker **client** sends commands to the Docker **daemon (dockerd)** over a REST API. The daemon manages images, containers, networks, and volumes. The daemon talks to a lower-level **container runtime** (containerd / runc) that actually creates and runs containers using Linux namespaces and cgroups. Images are pulled from and pushed to **registries** like Docker Hub.

Q: What is the difference between CMD and ENTRYPOINT?

A: Both define what runs when a container starts, but they behave differently when arguments are passed to *docker run*. **CMD** provides defaults that can be fully overridden. **ENTRYPOINT** defines the executable that always runs; anything you pass to *docker run* is appended as arguments. Best practice: use **ENTRYPOINT** for the binary, **CMD** for the default arguments.

Q: What are image layers and why do they matter?

A: Each instruction in a Dockerfile creates an immutable layer. Layers are cached, so if a layer hasn't changed, Docker reuses it on rebuilds — speeding up builds dramatically. Layers are also shared across images, saving disk space. This is why you order Dockerfile instructions from least-to most-frequently changing (e.g., copy package.json and install dependencies before copying your full source).

Q: What is a multi-stage build and why use it?

A: A multi-stage build uses multiple **FROM** statements in a single Dockerfile. You can build artifacts in one stage (with all the build tools) and copy only the final binaries into a clean runtime stage. The result is a much smaller, more secure production image because compilers, SDKs, and source code don't end up in the final image.

Q: Explain Docker networking modes.

A: Docker has several network drivers. **bridge** is the default — containers get a private IP and can talk to each other on the same host. **host** removes network isolation and the container uses the host's network stack directly. **none** disables networking. **overlay** spans multiple Docker hosts and is used by Swarm and Kubernetes. **macvlan** gives a container its own MAC address on the physical LAN.

Q: How do containers communicate with each other?

A: On the same host using a user-defined bridge network, containers can reach each other by their container name (Docker's built-in DNS). Across hosts, you use an overlay network or an orchestrator like Kubernetes. The default bridge network does not provide automatic DNS between containers, which is why creating a custom network is preferred.

Q: Difference between volumes and bind mounts?

A: **Volumes** are managed by Docker, stored in Docker's own area (/var/lib/docker/volumes), portable, and the recommended way to persist data. **Bind mounts** map a specific host directory into the container — they tie the container to the host's filesystem layout. Use volumes in production and bind mounts mainly for local development (e.g., mounting your source code).

Q: How do you reduce Docker image size?

A: Use a small base image (alpine, distroless). Use multi-stage builds to drop build dependencies. Combine RUN commands with && and clean up package caches in the same layer (e.g., `apt-get clean && rm -rf /var/lib/apt/lists/*`). Use a `.dockerignore` to exclude unnecessary files. Avoid installing recommended packages.

Q: What is the difference between EXPOSE and -p?

A: EXPOSE in the Dockerfile is purely **documentation** — it tells users which port the application listens on. It does not actually publish the port. `-p` in `docker run` (or `ports` in Compose) is what actually maps a container port to a host port and makes it reachable from outside.

Q: How do you manage secrets in Docker?

A: Never bake secrets into images. Options: environment variables passed at runtime (simple but visible in `docker inspect`), Docker secrets (in Swarm mode), mounted secret files from outside the container, BuildKit's `--secret` for build-time secrets, or — best — an external secret manager (HashiCorp Vault, AWS Secrets Manager) that the app reads at startup.

Q: What is a Docker daemon? Can you run Docker without it?

A: The Docker daemon (`dockerd`) is the background service that does all the actual work — building images, running containers, managing networks and volumes. Yes, there are daemonless alternatives like **Podman** that use a similar CLI but run containers without a central daemon, often as the regular user.

Q: How does Docker achieve isolation?

A: Docker leverages Linux kernel features: **namespaces** isolate what a process can see (PID, network, mount, user, etc.) and **cgroups** limit what resources it can use (CPU, memory, IO). Combined, they create the illusion of a separate machine even though the container is just a process on the host.

Q: What happens when you run `docker run nginx`?

A: Docker checks for the `nginx` image locally; if missing, it pulls it from the configured registry (Docker Hub by default). It creates a new writable layer on top of the image, sets up namespaces and cgroups, attaches the container to a network (default bridge), and starts the process defined by the image's CMD/ENTRYPOINT.

Q: How do you debug a failing container?

A: Run `docker logs <container>` to see stdout/stderr. Use `docker inspect` for configuration and exit code. If it's still running, `docker exec -it <container> sh` opens a shell. For a container that exits immediately, override the entrypoint: `docker run -it --entrypoint sh image` to poke around. Check `docker events` for system-level activity.

PART 2: KUBERNETES

Kubernetes (K8s) is an open-source container orchestration platform originally built by Google and now maintained by the CNCF. While Docker runs a container on one machine, Kubernetes manages thousands of containers across many machines, handling scheduling, scaling, self-healing, networking, and rolling updates.

2.1 Why Kubernetes?

- **Automated rollouts and rollbacks** — deploy new versions with zero downtime, roll back if something breaks.
- **Self-healing** — restarts failed containers, replaces unresponsive ones.
- **Horizontal scaling** — add/remove pods based on CPU, memory, or custom metrics.
- **Service discovery and load balancing** — built-in DNS and traffic distribution.
- **Storage orchestration** — automatically mount local or cloud storage.
- **Declarative configuration** — you describe the desired state in YAML, K8s makes it happen.
- **Cloud-agnostic** — runs the same way on AWS, GCP, Azure, on-prem, or your laptop.

2.2 Kubernetes Architecture

A Kubernetes cluster has two types of nodes: **Control Plane** (master) nodes that make global decisions, and **Worker** nodes that actually run your application workloads.

Control Plane Components

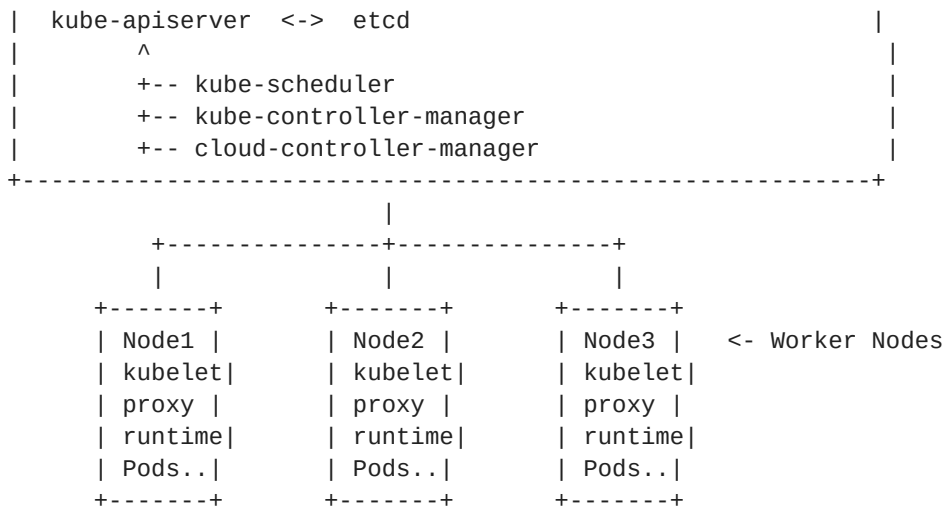
- **kube-apiserver**: The front door of the cluster. Every request (kubectl, kubelet, controllers) goes through it.
- **etcd**: Distributed key-value store. The single source of truth for cluster state.
- **kube-scheduler**: Watches for newly created pods and assigns them to nodes based on resources, affinity, taints, etc.
- **kube-controller-manager**: Runs controllers (Node, ReplicaSet, Deployment, Endpoint, etc.) that reconcile desired vs actual state.
- **cloud-controller-manager**: Integrates with cloud-provider APIs (load balancers, volumes, nodes).

Worker Node Components

- **kubelet**: Agent on each node. Talks to the API server, ensures containers described in pod specs are running and healthy.
- **kube-proxy**: Maintains network rules on each node, enabling Service abstraction (iptables/IPVS).
- **Container runtime**: Software that runs containers (containerd, CRI-O). Docker is no longer used directly since K8s 1.24.

Visual Summary

+----- CONTROL PLANE -----+



2.3 kubectl — The CLI

kubectl is the primary command-line tool to interact with the Kubernetes API. Almost every interview will test your fluency with it.

```

kubectl cluster-info           # Verify connectivity
kubectl get nodes              # List nodes
kubectl get pods               # List pods (default namespace)
kubectl get pods -n kube-system # Specific namespace
kubectl get pods -A            # All namespaces
kubectl get pods -o wide       # More details
kubectl get pods -o yaml       # Full YAML
kubectl describe pod mypod     # Detailed events + status
kubectl logs mypod             # Container logs
kubectl logs -f mypod -c sidecar # Follow specific container
kubectl exec -it mypod -- sh   # Shell into pod
kubectl apply -f deploy.yaml   # Create/update from YAML
kubectl delete -f deploy.yaml
kubectl rollout status deploy/web # Track rollout
kubectl rollout undo deploy/web  # Roll back
kubectl scale deploy/web --replicas=5
kubectl port-forward pod/mypod 8080:80 # Local testing

```

2.4 Pods

A **Pod** is the smallest deployable unit in Kubernetes. It wraps one or more containers that share network (same IP and port space) and storage. Most pods have a single container, but multi-container pods are used for tightly coupled helpers (sidecars).

Pod YAML Example

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx

```

```
image: nginx:1.25
ports:
- containerPort: 80
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi
```

You rarely create raw pods in production. Instead, you use higher-level controllers like **Deployments** that manage pods for you.

2.5 ReplicaSets and Deployments

A **ReplicaSet** ensures a specified number of identical pod replicas are running at all times. A **Deployment** wraps a ReplicaSet and adds declarative updates, rolling updates, and rollback support.

Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: myapp:1.2
        ports:
        - containerPort: 8080
```

Other Workload Controllers

- **StatefulSet**: For stateful apps that need stable network IDs and persistent storage (databases, Kafka).
- **DaemonSet**: Runs one copy of a pod on every (or selected) node. Used for log shippers, monitoring agents.
- **Job**: Runs a pod to completion (one-off tasks).
- **CronJob**: Runs a Job on a schedule (cron syntax).

2.6 Services

Pods are ephemeral — they get new IPs when restarted. A **Service** provides a stable virtual IP and DNS name to access a group of pods (selected by labels), and load-balances across them.

Service Types

Type	Behavior	Use Case
ClusterIP	Internal IP only (default)	Pod-to-pod inside the cluster
NodePort	Exposes on each node's IP at a static port (30000-32767)	Simple external access
LoadBalancer	Provisions a cloud load balancer	Production external access on cloud
ExternalName	Maps to an external DNS name (CNAME)	Reference external services

Service YAML

```
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: ClusterIP
  selector:
    app: web          # matches pod labels
  ports:
    - port: 80        # service port
      targetPort: 8080 # pod port
```

2.7 ConfigMaps and Secrets

ConfigMaps store non-sensitive configuration as key-value pairs. **Secrets** store sensitive data (passwords, tokens, certs) — they are base64-encoded (not encrypted by default; enable encryption at rest in etcd for real security).

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: info
  FEATURE_FLAG: enabled
---
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  password: c2VjcmV0MTIz  # base64 encoded
```

Consume them in a pod as env vars or as mounted files:

```
envFrom:
- configMapRef:
  name: app-config
- secretRef:
  name: db-secret
```


2.8 Namespaces

Namespaces provide logical isolation inside a cluster. They are used for environment separation (dev/staging/prod), team/project boundaries, and applying resource quotas and RBAC.

```
kubectl create namespace dev
kubectl get pods -n dev
kubectl config set-context --current --namespace=dev # set default
```

Default namespaces in every cluster: **default** (your stuff), **kube-system** (cluster components), **kube-public** (readable by all), **kube-node-lease** (node heartbeats).

2.9 Persistent Volumes

Containers are stateless, but apps often need durable storage. Kubernetes abstracts storage with:

- **PersistentVolume (PV)**: A piece of storage in the cluster (NFS, EBS, GCE disk). Cluster-level resource.
- **PersistentVolumeClaim (PVC)**: A user's request for storage of a certain size and access mode. Binds to a PV.
- **StorageClass**: Defines a class of storage and enables dynamic provisioning (PVs created on demand).

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: standard
  resources:
    requests:
      storage: 10Gi
```

2.10 Ingress

Ingress exposes HTTP/HTTPS routes from outside the cluster to services inside, using a single external entry point with hostname and path-based routing. Requires an **Ingress Controller** (nginx, Traefik, AWS ALB) installed in the cluster.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
spec:
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-svc
                port:
```

number : 80

2.11 Probes and Health Checks

Kubernetes uses probes to determine container health:

- **livenessProbe:** Is the container alive? If it fails, K8s restarts the container.
- **readinessProbe:** Is the container ready to serve traffic? If it fails, the pod is removed from Service endpoints (no traffic) but not restarted.
- **startupProbe:** Used for slow-starting apps. Disables liveness/readiness until it succeeds.

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  periodSeconds: 5
```

2.12 Kubernetes Interview Q&A;

L1 — Screening Level

Q: What is Kubernetes?

A: Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications across clusters of machines. It was originally created by Google and is now maintained by the CNCF.

Q: What is a Pod?

A: A pod is the smallest deployable unit in Kubernetes. It encapsulates one or more containers that share storage, network (same IP), and a specification on how to run. Most pods have a single container.

Q: What is a Node?

A: A node is a worker machine (VM or physical) in a Kubernetes cluster where pods run. Each node has a kubelet, a kube-proxy, and a container runtime.

Q: What is a Deployment?

A: A Deployment is a controller that manages a set of identical pods. It supports declarative updates, rolling updates, and rollbacks. You describe the desired state (e.g., 3 replicas of version 1.2) and the Deployment controller makes it so.

Q: What is a Service in Kubernetes?

A: A Service is an abstraction that defines a stable network endpoint to access a logical set of pods. Pods are ephemeral and get new IPs; the Service provides a stable virtual IP and DNS name and load-balances across the pods that match its selector.

Q: What is kubectl?

A: kubectl is the command-line tool to interact with a Kubernetes cluster. It sends requests to the API server to view, create, update, or delete resources.

Q: What is a Namespace?

A: A namespace is a virtual cluster inside a Kubernetes cluster. It provides logical isolation for resources, commonly used to separate environments (dev/staging/prod) or teams.

Q: What is the difference between a Pod and a Container?

A: A container is a single running process built from an image. A pod is the Kubernetes wrapper around one or more containers. The pod is what K8s schedules onto a node, and containers inside the same pod share the network and storage.

Q: What is the default Kubernetes namespace?

A: It is called *default*. All resources created without specifying a namespace go there.

Q: Name a few benefits of Kubernetes.

A: Self-healing, automatic horizontal scaling, declarative configuration, rolling updates with zero downtime, service discovery and load balancing, secret and config management, and cloud-agnostic deployment.

L2 — Technical Level

Q: Explain Kubernetes architecture.

A: A cluster has a **control plane** and one or more **worker nodes**. Control plane components: the **API server** (front door for all requests), **etcd** (key-value store, source of truth), **scheduler** (assigns pods to nodes), **controller-manager** (runs controllers that reconcile state), and the **cloud-controller-manager**. Each worker node runs **kubelet** (manages pods on the node), **kube-proxy** (implements Service networking), and a **container runtime** (containerd or CRI-O).

Q: What is the difference between a ReplicaSet and a Deployment?

A: A ReplicaSet ensures a specified number of pod replicas are running. A Deployment is a higher-level controller that manages ReplicaSets and adds rolling updates and rollbacks. In practice, you create Deployments — they create ReplicaSets for you. A Deployment update creates a new ReplicaSet alongside the old one to enable rolling deployments.

Q: Explain the four main Service types.

A: **ClusterIP** (default) gives a service an internal virtual IP reachable only inside the cluster. **NodePort** exposes the service on a static port on every node's IP — useful for development or when a cloud load balancer isn't available. **LoadBalancer** provisions an external cloud load balancer (AWS NLB/ALB, GCP LB) that forwards to the service — used in production on cloud providers. **ExternalName** maps the service to an external DNS name via CNAME, with no proxying.

Q: What is a StatefulSet and when do you use it?

A: A StatefulSet manages stateful applications that need stable, unique network identities (pod-0, pod-1...), stable persistent storage, and ordered, graceful deployment and scaling. Use it for databases (PostgreSQL, MongoDB), message brokers (Kafka), and other apps where pod identity matters.

Q: Difference between a ConfigMap and a Secret?

A: Both store key-value configuration. A ConfigMap is for non-sensitive data (log levels, feature flags). A Secret is for sensitive data (passwords, tokens). Secrets are base64-encoded — not encrypted by default. For real security, enable encryption at rest in etcd and/or use an external secret manager like Vault or AWS Secrets Manager.

Q: Explain liveness vs readiness probes.

A: Both check container health, but the consequence of failure differs. A failing **liveness** probe causes Kubernetes to restart the container — used to recover from deadlocks. A failing **readiness** probe causes the pod to be removed from Service endpoints, so no traffic is sent to it, but the container is not restarted — used during startup or when temporarily overloaded.

Q: What is the role of etcd?

A: etcd is the distributed key-value store that holds the entire cluster state — every object (pods, services, configmaps, secrets, etc.). It is the single source of truth. If you lose etcd, you lose the cluster. It must be backed up regularly and is typically run as a highly available cluster of 3 or 5 nodes.

Q: How does service discovery work in Kubernetes?

A: Kubernetes runs CoreDNS as an in-cluster DNS service. Every Service automatically gets a DNS name in the format `<svc>.<namespace>.svc.cluster.local`. Pods can resolve services by name. Additionally, kube-proxy programs iptables or IPVS rules on each node so traffic to a Service's virtual IP is load-balanced to the pods behind it.

Q: What is an Ingress and how does it differ from a LoadBalancer?

A: A LoadBalancer service exposes **one** service externally and usually provisions a dedicated cloud load balancer. An Ingress is a smarter Layer 7 entry point: it can route HTTP/HTTPS traffic to **many** services based on hostname and URL path. You need an Ingress Controller (NGINX, Traefik, AWS ALB Controller) running in the cluster for Ingress resources to take effect.

Q: How do rolling updates work?

A: When you update a Deployment, the controller creates a new ReplicaSet for the new pod template and gradually scales it up while scaling the old ReplicaSet down. The *maxSurge* and *maxUnavailable* values control how many extra pods can be added and how many can be unavailable during the rollout. If anything goes wrong, *kubectl rollout undo* reverts to the previous ReplicaSet.

Q: What are taints and tolerations?

A: Taints are applied to nodes to repel pods. Tolerations are applied to pods to allow them to be scheduled onto tainted nodes. Together they let you keep certain pods off certain nodes — for example, dedicated GPU nodes that should only run ML workloads.

Q: What is the difference between requests and limits?

A: Requests are the resources guaranteed to a container — the scheduler uses them to decide which node has room. **Limits** are the maximum the container can use; if it exceeds the memory limit, it's OOM-killed; CPU is throttled. Always set realistic requests (for scheduling) and limits (for stability).

Q: How does Kubernetes self-heal?

A: Controllers continuously compare actual state (in etcd, via the API) to desired state. If a pod crashes, the ReplicaSet creates a replacement. If a node goes down, pods on it are rescheduled elsewhere. If a liveness probe fails, the container is restarted. All of this happens automatically through the control loop pattern.

Q: What is Helm?

A: Helm is the package manager for Kubernetes. A Helm **chart** bundles related Kubernetes YAML files (Deployment, Service, ConfigMap, Ingress, etc.) into a reusable, versioned, and parameterizable package. Helm makes it easy to install complex applications (Prometheus, NGINX Ingress, cert-manager) with a single command.

PART 3: TERRAFORM

Terraform is the industry-standard tool for **Infrastructure as Code (IaC)**. It lets you describe cloud infrastructure (servers, databases, networks, DNS, IAM) in a declarative configuration language called HCL, then create and manage it with a few commands.

3.1 Infrastructure as Code (IaC)

Infrastructure as Code means defining your infrastructure in machine-readable files instead of clicking through cloud consoles. Benefits include:

- **Version control** — infrastructure changes are tracked in Git, with PRs and review.
- **Reproducibility** — recreate identical environments on demand.
- **Automation** — provision infrastructure in seconds, not days.
- **Consistency** — eliminate configuration drift between environments.
- **Collaboration** — teams can work together on infrastructure.

3.2 What is Terraform?

Terraform, created by HashiCorp, is a cloud-agnostic IaC tool. You describe your desired infrastructure in **HCL** (HashiCorp Configuration Language), and Terraform figures out the API calls needed to make it real.

Terraform vs Alternatives

Tool	Approach	Scope
Terraform	Declarative, cloud-agnostic	Provision any infrastructure (multi-cloud)
AWS CloudFormation	Declarative	AWS only
Azure ARM/Bicep	Declarative	Azure only
Pulumi	Imperative (real languages)	Multi-cloud, code in Python/TS/Go
Ansible	Imperative + procedural	Config management, can also provision

3.3 Terraform Workflow

The core Terraform workflow has four commands you will run constantly:

Command	What it does
terraform init	Downloads providers, initializes the backend, prepares working dir
terraform plan	Shows what Terraform WILL do — diff between code and real infrastructure
terraform apply	Actually creates/updates/destroys resources to match the code
terraform destroy	Tears down everything Terraform manages

Other Useful Commands

```
terraform validate    # Check syntax
terraform fmt         # Auto-format .tf files
terraform show        # Display current state in human form
```

```
terraform state list  # List all resources in state
terraform output      # Show output values
terraform import      # Bring an existing resource under Terraform's management
terraform refresh     # Update state with real infrastructure
```

3.4 Providers and Resources

A **provider** is a plugin that knows how to talk to a specific API (AWS, Azure, GCP, Kubernetes, GitHub, Datadog, etc.). A **resource** is an infrastructure object managed by a provider (an EC2 instance, an S3 bucket, a DNS record).

Example: Provision an AWS EC2 Instance

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "web" {
  ami          = "ami-0abcdef1234567890"
  instance_type = "t3.micro"

  tags = {
    Name = "WebServer"
    Env  = "dev"
  }
}
```

Data Sources

A **data source** reads information from a provider without managing it — for example, looking up the latest Amazon Linux AMI:

```
data "aws_ami" "latest_amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

resource "aws_instance" "web" {
  ami          = data.aws_ami.latest_amazon_linux.id
  instance_type = "t3.micro"
}
```

3.5 Variables, Outputs, Locals

Input Variables

```
# variables.tf
variable "region" {
  description = "AWS region"
  type        = string
  default     = "ap-south-1"
}

variable "instance_count" {
  type    = number
  default = 2
}

variable "tags" {
  type = map(string)
  default = {
    Env = "dev"
    Team = "platform"
  }
}
```

You can override variables via:

- Command line: *terraform apply -var="region=us-east-1"*
- *terraform.tfvars* file (auto-loaded)
- **.auto.tfvars* files
- Environment variables: *TF_VAR_region=us-east-1*

Outputs

Outputs expose values from your configuration — useful for showing IPs after creation, or passing values between modules.

```
output "instance_public_ip" {
  description = "Public IP of the web server"
  value       = aws_instance.web.public_ip
}
```

Locals

Locals are named expressions you can reuse — like local variables inside a Terraform configuration.

```
locals {
  common_tags = {
    Environment = var.environment
    ManagedBy   = "terraform"
    Project     = "webapp"
  }
}

resource "aws_s3_bucket" "data" {
  bucket = "my-data-bucket"
  tags   = local.common_tags
}
```

3.6 Terraform State

Terraform stores the mapping between your configuration and real-world infrastructure in a **state file** (*terraform.tfstate*). This is critical — without state, Terraform doesn't know what it already created.

Why State Matters

- Maps configuration to real resources (resource A in code → resource ID in cloud).
- Tracks metadata (dependencies, attributes).
- Improves performance — Terraform doesn't have to query every resource on every run.
- Enables collaboration when stored remotely with locking.

Local vs Remote State

By default, state is stored locally in *terraform.tfstate*. This is fine for solo experimentation but **dangerous for teams** — two people running apply at the same time can corrupt state.

In production, use a **remote backend** with state locking: AWS S3 + DynamoDB lock table, Terraform Cloud, Azure Blob, GCS, or HashiCorp Consul.

```
terraform {
  backend "s3" {
    bucket      = "my-tf-state"
    key         = "prod/terraform.tfstate"
    region      = "ap-south-1"
    dynamodb_table = "tf-state-lock"
    encrypt     = true
  }
}
```

Never commit state files to Git. They often contain secrets in plain text (passwords, keys). Add **.tfstate** to *.gitignore*.

3.7 Modules

A **module** is a reusable, parameterized set of Terraform configurations — like a function in programming. Modules can be local folders, Git repositories, or published in the Terraform Registry.

Using a Module

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "5.1.0"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs          = ["ap-south-1a", "ap-south-1b"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]

  enable_nat_gateway = true
}
```

Writing Your Own Module

A typical module structure:

```
modules/
  ec2-instance/
    main.tf      # resources
    variables.tf # input variables
    outputs.tf   # outputs
    README.md
```

Consume it from the root configuration:

```
module "web_server" {
  source      = "../modules/ec2-instance"
  instance_type = "t3.medium"
  ami        = "ami-xxxx"
}
```

3.8 Workspaces

Workspaces allow you to manage multiple state files within the same configuration — typically for environment separation (dev, staging, prod) without duplicating code.

```
terraform workspace list
terraform workspace new dev
terraform workspace new prod
terraform workspace select prod
terraform workspace show
```

Use `terraform.workspace` in code to make environment-specific decisions. **Caveat:** for serious environment separation, many teams prefer separate directories or repositories per environment rather than workspaces, because workspaces share the same backend configuration.

3.9 Provisioners and Lifecycle

Provisioners

Provisioners run scripts on a local or remote machine as part of resource creation/destruction. Examples: *local-exec*, *remote-exec*, *file*. HashiCorp recommends them only as a last resort — prefer cloud-native tools like `user_data`, Ansible, or configuration management.

```
resource "aws_instance" "web" {
  # ...
  provisioner "local-exec" {
    command = "echo ${self.public_ip} > ip_address.txt"
  }
}
```

Lifecycle Meta-Arguments

```
resource "aws_instance" "web" {
  # ...
  lifecycle {
    create_before_destroy = true # avoid downtime on replacement
    prevent_destroy       = true # block accidental destroy
    ignore_changes        = [tags] # ignore drift in these attributes
  }
}
```

3.10 Best Practices

- Use **remote state** with locking for any team work.
- Use **modules** to keep code DRY and reusable.
- **Pin provider versions** with *required_providers* to avoid breaking changes.
- **Format and validate** on every commit: *terraform fmt + terraform validate*.
- Never commit **.tfstate** or **.tfvars** with secrets to Git. Add to *.gitignore*.
- Use **workspaces or separate directories** for environment isolation.
- Always run **plan** before **apply** — read the plan carefully.
- Use **variables and locals** instead of hardcoding values.
- Tag all resources consistently for cost tracking and ownership.
- Use **data sources** instead of hardcoded IDs whenever possible.
- Integrate with CI/CD (*terraform plan* on PRs, *apply* after merge).
- Run **tflint** and **tfsec** for linting and security scanning.

3.11 Terraform Interview Q&A;

L1 — Screening Level

Q: What is Terraform?

A: Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp. It lets you define cloud infrastructure in HCL configuration files and provision it through a consistent CLI workflow. It supports many providers (AWS, Azure, GCP, Kubernetes, and hundreds more).

Q: What is Infrastructure as Code?

A: It is the practice of managing and provisioning infrastructure through code instead of manual processes. Benefits include version control, repeatability, automation, and consistency across environments.

Q: What language does Terraform use?

A: HCL — HashiCorp Configuration Language. It is human-readable and designed specifically for describing infrastructure. Terraform also accepts JSON as an alternative format.

Q: What are the main Terraform commands?

A: *terraform init* sets up the working directory and downloads providers. *terraform plan* shows what changes will be made. *terraform apply* actually creates or updates resources. *terraform destroy* tears them down.

Q: What is a provider?

A: A provider is a plugin that lets Terraform talk to a particular API. For example, the AWS provider knows how to create EC2 instances, S3 buckets, and IAM roles by calling AWS APIs.

Q: What is a resource?

A: A resource is a single piece of infrastructure managed by Terraform — an EC2 instance, a VPC, an S3 bucket, a DNS record, a Kubernetes deployment, etc. Defined with a *resource* block in HCL.

Q: What is the Terraform state file?

A: *terraform.tfstate* is a JSON file Terraform uses to map your configuration to real-world resources. It tracks what has been created so Terraform knows what to update or destroy on the next run.

Q: What is a Terraform module?

A: A module is a reusable container of Terraform configurations. It can take input variables and produce outputs — like a function. Modules can be local folders, Git repos, or pulled from the public Terraform Registry.

Q: Is Terraform declarative or imperative?

A: Declarative. You describe the desired end state of your infrastructure, and Terraform figures out the steps needed to get there — instead of you writing step-by-step instructions.

Q: What does *terraform init* do?

A: It initializes the working directory: downloads the required providers, sets up the backend (where state will be stored), and installs any modules referenced in the configuration. It must be run before any other command.

L2 — Technical Level

Q: Walk me through the Terraform workflow.

A: First, write your configuration in `.tf` files. Run `terraform init` to download providers and set up the backend. Run `terraform validate` and `terraform fmt` for syntax and formatting checks. Run `terraform plan` to see what changes Terraform will make and verify they're what you expect. Run `terraform apply` to execute. Terraform compares desired state (code) with current state (state file), computes a diff, and calls provider APIs to bring the actual infrastructure in line.

Q: Why is state important and how do you secure it?

A: State is Terraform's source of truth — it maps configuration to real resources and tracks attributes. Without it, Terraform doesn't know what already exists. Secure it by (1) storing it in a remote backend like S3 with encryption at rest, (2) enabling state locking with a DynamoDB table or equivalent to prevent concurrent applies, (3) never committing it to Git, (4) restricting IAM access to the backend, and (5) using `sensitive = true` on outputs that contain secrets.

Q: What is state locking?

A: State locking prevents two people from running `terraform apply` at the same time, which could corrupt the state file. The S3 + DynamoDB backend uses a DynamoDB table to hold a lock during operations; Terraform Cloud handles locking automatically.

Q: How do you handle multiple environments (dev, staging, prod)?

A: Two main patterns. **Workspaces:** one configuration, separate state per workspace, use `terraform.workspace` in code. Simple but shares the same backend config. **Separate directories or repos per environment:** more isolation, different backends, different variables. Most production teams prefer separate directories with shared modules — it gives stronger blast-radius control.

Q: Difference between a variable and a local?

A: A **variable** is an input to your configuration — provided by the caller (CLI, tfvars file, env var, or parent module). A **local** is a named expression computed inside the configuration, used to avoid repetition. You can't override locals from outside; they exist only for clarity and reuse.

Q: How do you import an existing resource into Terraform?

A: Use `terraform import <resource_address> <real_id>` — for example, `terraform import aws_instance.web i-0abc....`. This brings the resource into state, but you still need to write matching HCL configuration manually. Terraform 1.5+ also supports an `import` block in code so the import is reviewable as part of a plan.

Q: What is the difference between count and for_each?

A: Both create multiple instances of a resource. **count** uses a number — instances are indexed by integer (`resource[0]`, `resource[1]`). **for_each** uses a map or set — instances are keyed by string (`resource["web"]`). Use `for_each` when items have stable identifiers; otherwise removing an item in the middle of a count list shifts everything and Terraform will destroy and recreate the wrong resources.

Q: Explain the lifecycle block.

A: Lifecycle meta-arguments customize how Terraform handles resource changes. `create_before_destroy = true` creates a replacement before destroying the old one (good for

zero-downtime). `prevent_destroy = true` blocks *terraform destroy* on critical resources. `ignore_changes = [tags]` tells Terraform not to revert changes made outside Terraform to those attributes.

Q: How does Terraform handle dependencies?

A: Mostly automatically. When one resource references another's attribute (e.g., `aws_subnet.example.id`), Terraform infers a dependency and orders operations correctly. For dependencies that can't be inferred from references, use the explicit `depends_on` meta-argument.

Q: What is the difference between Terraform and Ansible?

A: Terraform is primarily for **provisioning** infrastructure — creating cloud resources from scratch — and is declarative. Ansible is primarily for **configuration management** — installing software, configuring services on existing machines — and is mostly imperative. They are often used together: Terraform creates the servers, Ansible configures what runs on them.

Q: What happens if you run `terraform apply` twice in a row with no changes?

A: Terraform refreshes the state, compares it to the configuration, finds no differences, and reports *No changes. Your infrastructure matches the configuration.* Nothing is modified. This idempotency is a core property of Terraform.

Q: What is drift and how do you handle it?

A: **Drift** is when real infrastructure changes outside Terraform — someone clicks in the AWS console and modifies a resource. Detect drift with `terraform plan` — it will show the difference. Resolve by either reverting the manual change (let Terraform reset it on the next apply) or updating the Terraform code to reflect the new reality.

Q: What is the Terraform Registry?

A: It is HashiCorp's public catalog at registry.terraform.io for providers and modules. You can find official AWS, Azure, GCP, and Kubernetes providers, plus thousands of community modules for common patterns (VPC, EKS, RDS, etc.). Private registries also exist (Terraform Cloud, internal company registries).

Q: Can Terraform manage Kubernetes resources?

A: Yes — the Kubernetes provider lets you create Deployments, Services, ConfigMaps, etc. directly from Terraform. Some teams use it; others prefer to keep Kubernetes manifests separate (in Helm charts or GitOps tools like ArgoCD/Flux) and use Terraform only to provision the cluster itself.

PART 4: INTERVIEW STRATEGY

4.1 How to Answer L1 vs L2 Questions

L1 (Screening) Strategy

L1 is usually a recruiter or junior engineer doing a fit check. They are looking for **basic understanding**, not depth. Keep your answers:

- **Short (2-3 sentences).** One clean definition + a one-line example.
- **Confident.** If you don't know, say so briefly and move on.
- **Free of jargon overload.** A clear simple answer beats a buzzword salad.
- **Connected to use cases.** Don't just define — say why it exists.

L2 (Technical) Strategy

L2 is a senior engineer or tech lead. They want to see how you **think**, not just what you memorized.

- **Structure your answer.** Define → mechanism → example → trade-offs.
- **Mention trade-offs.** Senior engineers love hearing "the downside of X is Y."
- **Use real examples** from your projects when possible.
- **Draw it if possible.** Architecture questions are easier on a whiteboard.
- **Ask clarifying questions** for scenario questions before diving in.
- **Admit uncertainty** when relevant — "I haven't used X in production but I'd start by..." is fine.

4.2 Scenario-Based Cross-Topic Questions

Q: How would you deploy a Node.js application end-to-end?

A: I would write a multi-stage Dockerfile producing a small alpine-based image. Build and push it in CI/CD to a registry like ECR. Use Terraform to provision the underlying infrastructure: VPC, EKS cluster, RDS database, IAM roles. Write Kubernetes manifests (Deployment, Service, Ingress, ConfigMap, Secret) — possibly bundled as a Helm chart. Apply them via *kubectl apply* or a GitOps tool like ArgoCD. Add liveness/readiness probes, resource requests/limits, and a Horizontal Pod Autoscaler. Front it with an Ingress and TLS via cert-manager.

Q: A pod keeps crashing with CrashLoopBackOff. How do you debug?

A: Start with *kubectl describe pod <name>* to see events — often you'll find image pull errors, OOMKilled, or failed probes right there. Then *kubectl logs <pod>* for app errors, and *kubectl logs --previous* for the previous crashed container. Check resource limits — memory limit too low causes OOMKill. Check liveness probe — too aggressive a probe restarts a healthy app. Try running the same image locally with Docker to isolate whether it's an app bug or a Kubernetes config issue.

Q: Your Terraform apply failed halfway. What now?

A: First, do not panic — Terraform marks resources as *tainted* or partially created in the state. Read the error carefully — it usually tells you what failed (quota, permission, naming conflict). Fix the root cause. Run *terraform plan* again — Terraform will show what still needs to happen. Run

apply again. If state got corrupted, you can use *terraform state rm* to remove a problematic entry and re-import, or restore from a backup of the remote state.

Q: How do you do zero-downtime deployment of a new container image?

A: Use a Kubernetes Deployment with the RollingUpdate strategy and set *maxSurge* and *maxUnavailable* appropriately (e.g., *maxSurge*=1, *maxUnavailable*=0). Make sure pods have proper readiness probes so traffic only goes to ready pods. Update the image tag in the Deployment and *kubectl apply*. Kubernetes will spin up new pods, wait for readiness, then terminate old pods one by one. Monitor with *kubectl rollout status*. If something is wrong, *kubectl rollout undo* reverts instantly.

Q: How would you store database credentials for an application?

A: Never put them in code or in the Docker image. Best options: (1) Kubernetes Secrets with encryption at rest in etcd, mounted as env vars or files. (2) An external secret manager like AWS Secrets Manager, Vault, or GCP Secret Manager — accessed at startup using the pod's IAM/IRSA role. (3) For Terraform, use the relevant data source to fetch the secret at runtime; never hardcode it. Always rotate credentials regularly.

Q: What is the difference between Docker Compose and Kubernetes?

A: Compose is a tool for running multi-container apps on **a single host** — typically for local development or small deployments. Kubernetes is a full orchestration platform for running containers **across a cluster of many hosts**, with scheduling, self-healing, auto-scaling, rolling updates, and production-grade networking. Compose is fast and simple; Kubernetes is powerful and complex. You typically use Compose locally and Kubernetes in production.

4.3 Final Tips

Two Weeks Before the Interview

- Set up a small **hands-on project**: containerize an app, deploy it to a local Kubernetes cluster (kind, minikube, or Docker Desktop), and provision a tiny AWS/GCP environment with Terraform.
- Practice *kubectl* commands daily — fluency matters.
- Read the official documentation for each tool — short, well-written tutorials.
- Use Katacoda / Play with Docker / Killer.sh for browser-based labs.

The Day Before

- Re-read this PDF's Q&A; sections.
- Re-check Dockerfile, Kubernetes manifest, and Terraform syntax cheat sheets.
- Prepare 1-2 stories from your own projects/internships that touch each tool.
- Sleep well — fatigue kills technical reasoning.

During the Interview

- **Listen carefully** to the question. Many candidates answer the wrong question.
- **Think out loud.** Interviewers can't see your thoughts — share them.
- **Ask clarifying questions** for ambiguous scenarios.
- **It's fine to say "I don't know."** Bluffing is far worse — they will catch it.
- **Tie your answers to your experience** when possible — it makes you memorable.
- **Have your own questions ready** — about the team, the stack, on-call practices, learning opportunities.

Final Words

Docker, Kubernetes, and Terraform together cover the modern cloud infrastructure stack. As an associate software engineer, you are not expected to be an expert — interviewers want to see that you understand the **concepts**, can use the basic **commands**, and can **reason** about real problems. Build a small project, deploy it end-to-end, break it on purpose, and fix it. Hands-on experience beats any amount of memorization.

Good luck with your interviews!